

class120 树的重心 笔记

树上问题专题三。前置：链式前向星建图（讲解059）、树形DP（讲解078/079）、递归改迭代（讲解118）。

树的重心的定义与性质

三种定义（虽然定义不同，但求出来的重心是同一个，或同两个）：

1. 以某个节点为根时，**最大子树的节点数最少**，该节点是重心；
2. 以某个节点为根时，**每棵子树的节点数都不超过总节点数的一半**，该节点是重心；
3. **所有节点走向该节点的总距离和最小**，该节点是重心。

补充性质：

4. 一棵树**最多有两个重心**；若有两个，一定通过一条边相邻；
5. 增加/删除一个**叶节点**后，重心要么不动，要么最多移动一条边；
6. 两棵树连起来后，新树的重心一定在原来两棵树重心的**路径**上；
7. 边权都为正时，所有节点汇聚到**重心**走过的总距离最小（与边权怎么分布无关）。

题目一：平衡行为（POJ 1655） — 重心求解方式一

题面

n 个节点、 $n-1$ 条边的树。返回重心点编号，以及该重心最大子树的节点数；若重心不唯一，返回编号较小的那个。

核心思路

用第一种定义求解。从 1 号点开始 DFS，收集每个节点 u 的 $size[u]$ （以 1 为根时 u 这棵子树的节点数）。以 u 为根时，它分成的“子树”包括：

- 每个孩子 v 对应的子树 $size[v]$ ；
- 加上“上面那一块”： $n - size[u]$ 。

取这些值的最大值 $maxsub$ ，就是 u 做根时最大子树的节点数。全局维护 $maxsub$ 最小的节点（并列取编号更小者）即为重心。

坑点

- 别忘了“上面那一块” $n - size[u]$ ，很多人漏掉。
- 重心可能有两个，题目要求返回编号较小者，需要 `maxsub < best || (maxsub == best && u < center)` 的判断。

代码

```
1  const int MAXN = 20001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int size[MAXN], center, best;
5
6  void dfs(int u, int f) {
7      size[u] = 1;
8      // 以当前节点u做根节点，最大的子树有多少节点
```

```

9     int maxsub = 0;
10    for (int e = head[u], v; e != 0; e = nxt[e]) {
11        v = to[e];
12        if (v != f) {
13            dfs(v, u);
14            size[u] += size[v];
15            maxsub = max(maxsub, size[v]);
16        }
17    }
18    maxsub = max(maxsub, n - size[u]);
19    // 题目要求找到编号最小的重心
20    if (maxsub < best || (maxsub == best && u < center)) {
21        best = maxsub;
22        center = u;
23    }
24 }

```

题目二：教父 (POJ 3107) — 重心求解方式二

题面

n 个节点、 $n-1$ 条边的树。返回**所有**重心点（最多两个）；一个重心打印一个数，两个重心打印 `a b`（空格分隔）。

核心思路

用第二种定义求解。DFS 收集每个节点的 `size[u]` 和 `maxsub[u]`（以 u 为根时最大子树的节点数，同样 = 孩子子树最大值与 $n - \text{size}[u]$ 的较大者）。最后统一扫一遍：`maxsub[i] <= n / 2` 的节点就是重心。因为重心最多两个，用一个长度为 2 的数组收集即可。

坑点

- 和方式一一样，`maxsub[u]` 的"上面那块"是 $n - \text{size}[u]$ ，不能漏。
- 方式二不需要全局 `best/center`，每个节点各自存 `maxsub`，扫一遍判定即可，思路更直接。

代码

```

1  const int MAXN = 50001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int size[MAXN], maxsub[MAXN];
5
6  void dfs(int u, int f) {
7      size[u] = 1;
8      maxsub[u] = 0;
9      for (int e = head[u], v; e != 0; e = nxt[e]) {
10         v = to[e];
11         if (v != f) {
12             dfs(v, u);
13             size[u] += size[v];
14             maxsub[u] = max(maxsub[u], size[v]);
15         }
16     }
17     maxsub[u] = max(maxsub[u], n - size[u]);
18 }

```

题目三：牛群聚集（P2986）——带点权的重心

题面

n 个农场（节点），每个农场有若干头牛（点权）， $n-1$ 条带权值的双向边（边权）。所有牛要汇聚到同一个点，求走过的总距离最小是多少（返回总距离，不需要返回汇聚点）。

核心思路

边权都是正数，由性质 7：所有牛汇聚到重心时总距离最小。所以两步：

1. 求带点权的重心：把 size 的含义从"节点数"换成"牛的总量"——
 - size[u] 初始值不再是 1，而是 cow[u]（本农场牛数）；
 - 最后"上面那块" = cowSum - size[u]（牛的总数减去 u 子树牛量）。其余与普通重心求法一致（用第一种定义，维护最大子树牛量最小的点）。
2. 算总距离：找到重心 center 后，从 center 出发 DFS 得到每个点 path[i]（到重心的距离），答案 = $\sum \text{cow}[i] * \text{path}[i]$ 。

坑点

- 递归版会爆栈（ n 可达 10^5 ），必须改成迭代版（自己压栈模拟递归，见讲解118）。findCenter 里"先 DFS 收集完子树，再统一统计"的写法就是为了方便改迭代。
- size 初始值 = cow[u]，不是 1；"上面那块"是 cowSum - size[u]，不是 $n - \text{size}[u]$ 。
- 答案要用 long long（牛数 × 距离可能溢出 int）。

代码（递归版核心）

```

1  const int MAXN = 100001;
2  int n, cnt, cowSum, best, center;
3  int cow[MAXN], head[MAXN], nxt[MAXN << 1], to[MAXN << 1], weight[MAXN << 1];
4  int size[MAXN]; // size[i] : 以i为头的子树, 牛的总量
5  int path[MAXN]; // path[i] : 从重心到达i的距离
6
7  void findCenter(int u, int f) {
8      size[u] = cow[u];
9      for (int e = head[u], v; e != 0; e = nxt[e]) {
10         v = to[e];
11         if (v != f) findCenter(v, u);
12     }
13     // 遍历完成后再统一统计（为了后续改迭代版方便）
14     int maxsub = 0;
15     for (int e = head[u], v; e != 0; e = nxt[e]) {
16         v = to[e];
17         if (v != f) {
18             size[u] += size[v];
19             maxsub = max(maxsub, size[v]);
20         }
21     }
22     maxsub = max(maxsub, cowSum - size[u]);
23     if (maxsub < best) { best = maxsub; center = u; }
```

```

24 }
25
26 void setPath(int u, int f) {
27     for (int e = head[u], v; e != 0; e = nxt[e]) {
28         v = to[e];
29         if (v != f) { path[v] = path[u] + weight[e]; setPath(v, u); }
30     }
31 }
32
33 long long compute() {
34     for (int i = 1; i <= n; i++) cowSum += cow[i];
35     findCenter(1, 0);
36     path[center] = 0;
37     setPath(center, 0);
38     long long ans = 0;
39     for (int i = 1; i <= n; i++) ans += (long long)cow[i] * path[i];
40     return ans;
41 }

```

迭代版：用 `int ufe[MAXN][3]`（存 `u`、`f`、`e`）作为手写栈，`push/pop` 模拟递归的进入与回溯，逻辑与递归版完全一致。

题目四：删增边使其重心唯一（CF1406C）

题面

n 个节点、 $n-1$ 条边的树。允许先删一条边、再增加一条边，使重心唯一。打印任意一种方案（两行：删的边 `u v`、加的边 `x y`）。

核心思路

先 DFS 求出每个节点最大子树大小 `maxsub`，统计重心数量：

- **只有一个重心**：删掉连接重心的任意一条边，再把这条边原样加回（打印同一条边两次即可）。
- **有两个重心**（设为 `centers[0]`、`centers[1]`）：选定 `centers[0]` 作为唯一重心；在另一个重心 `centers[1]` 里随便找一个叶子，删掉叶子与其父亲的边，把叶子挂到 `centers[0]` 下。

坑点

- 两个重心一定相邻；找叶子时把 `centers[0]` 当作 `centers[1]` 的父亲，从 `centers[1]` 一路往下找，**找到一个叶子就 return**，不用遍历整棵树。
- 只有一个重心时打印同一条边两次，这是题目（已实测）的要求。

代码

```

1  const int MAXN = 100001;
2  int n, cnt;
3  int head[MAXN], nxt[MAXN << 1], to[MAXN << 1];
4  int size[MAXN], maxsub[MAXN];
5  int centers[2], leaf, leafFather;
6
7  void dfs(int u, int f) {
8      size[u] = 1; maxsub[u] = 0;
9      for (int e = head[u], v; e != 0; e = nxt[e]) {

```

```

10     v = to[e];
11     if (v != f) {
12         dfs(v, u);
13         size[u] += size[v];
14         maxsub[u] = max(maxsub[u], size[v]);
15     }
16 }
17 maxsub[u] = max(maxsub[u], n - size[u]);
18 }
19
20 // 随意找一个叶节点和它的父亲（哪一组都行）
21 void find(int u, int f) {
22     for (int e = head[u]; e != 0; e = nxt[e]) {
23         if (to[e] != f) { find(to[e], u); return; }
24     }
25     leaf = u; leafFather = f;
26 }
27
28 int centerCnt() {
29     int m = 0;
30     for (int i = 1; i <= n; i++)
31         if (maxsub[i] <= n / 2) centers[m++] = i;
32     return m;
33 }
34 // 主逻辑: cnt==1 打印 centers[0] 与其一条邻边两次;
35 // cnt==2 时 find(centers[1], centers[0]), 打印 (leafFather, leaf) 与
    (centers[0], leaf)。

```